

28. Instagram (Photo & Video Sharing)

Interviewer: Let's design Instagram. Users take a photo or video, upload it, add a caption, and it becomes visible to their followers — who can like and comment on it. There are also Stories, which disappear after 24 hours. Design the backend.

Candidate: Great — and I want to be upfront about scope, because "design Instagram" spans at least two very different hard problems: **(1) moving and serving huge binary media at global scale**, and **(2) fanning a post out to followers' feeds**. The second one — the timeline/fan-out problem — is basically the News Feed design, and I'd rather not re-derive it from scratch here. I'll touch it briefly, but I want to spend most of our time on what's actually unique to a photo/video app: the **upload pipeline, blob storage, transcoding, and CDN delivery** — because that's where the real engineering is.

Interviewer: Agreed, lean into media. Go ahead and clarify.

1. Clarifying the requirements

Candidate: A few questions: 1. Photos and videos both — what's the size range? Phone photos (~5–10MB), but also multi-minute 4K video? 2. Do we need multiple resolutions per asset (thumbnail, feed-size, full-res, video ABR ladder), or just store-and-serve-as-is? 3. Stories — 24h TTL, viewed-once-per-viewer tracking needed? 4. Is strict global consistency required (e.g., must a like count be exact), or is "close enough, converges quickly" acceptable? 5. Scale — are we talking Instagram-scale (billions of photos, hundreds of millions of DAU)?

Interviewer: Photos up to ~15MB, videos up to a couple minutes at up to 4K. Yes — we need a thumbnail, a feed-optimized version, and (for video) an adaptive bitrate ladder. Stories: 24h TTL, and we do want per-viewer view tracking. Like/comment counts can be eventually consistent — a like count off by a few for a second is fine. Assume Instagram scale: ~500M DAU, ~100M photos/videos uploaded per day.

Candidate: Then here's what I'm designing to.

Functional - `POST /media` → upload a photo/video, attach caption, get back a `postId`. - Generate and serve a **thumbnail**, a **feed-resolution** version, and (video) an **ABR ladder**, all derived asynchronously from the original. - `GET /feed` → timeline of posts from followed accounts (leaning on the News Feed design — doc 04 — for the fan-out mechanics; I'll only note where media changes that picture). - Like / comment on a post; view counts. - Post a **Story**; it expires and disappears after 24h; track per-viewer "seen" state.

Non-functional - **Durability of media is non-negotiable** — losing someone's photo is a trust-destroying bug, unlike losing a like. - **Massive read amplification per byte written**: one photo is uploaded once, then viewed by thousands of followers, over and over, from every part of the globe. This

is the defining shape of the problem — **write once, read (from everywhere) a million times**. - **Low latency, globally**, for media *delivery* specifically — a slow-loading image feels broken. - **Upload must tolerate flaky mobile networks** — large files, weak wifi, users walking around. - **Processing (thumbnails/transcodes) is asynchronous** — the user shouldn't stare at a spinner for 30 seconds while we transcode a video. - **Counters (likes/comments/views) are high-write, low-consistency-requirement** — perfect eventual-consistency candidates.

2. Estimation — why this is a storage-and-fanout-of-bytes problem

500M DAU. ~100M media uploads/day (photos + videos).

-> $100,000,000 / 86,400 \approx 1,160$ uploads/sec avg (peak several x that at "prime scrolling hours" — evenings, launches).

Avg photo (post-compression) ~ 200KB feed version + ~2-8MB original kept for reprocessing.

Avg video: ~50MB original, transcoded to a ~5-rung ABR ladder (~150MB total variants).

Say 80% photo, 20% video uploads:

80M photos/day * ~3MB avg original = 240 TB/day

20M videos/day * ~50MB avg original = 1 PB/day

~1.25 PB/day of NEW media, growing forever -> object storage, not a database, obviously.

THE NUMBER THAT MATTERS: reads vastly outnumber writes.

500M DAU * ~150 feed items viewed/day = 75B media views/day

-> $75,000,000,000 / 86,400 \approx 870,000$ media reads/sec average, bursty far higher.

So: ~1,000 writes/sec of large blobs, vs. ~1,000,000 reads/sec of (mostly small, cached) images/video segments, from users on every continent. This is a "store once, deliver at global CDN scale" problem, not a database-throughput problem.

Candidate: That read:write ratio (roughly 1000:1, and the reads are latency-sensitive and geographically scattered) is what pushes every design decision toward **object storage + CDN**, not toward scaling a database.

3. A simple V1 (and why it breaks immediately)

[Mobile App] --POST /media (multipart, raw bytes)--> [App Server] --> [Postgres: BLOB column]

└─> generates thumbnail synchronously, in-request

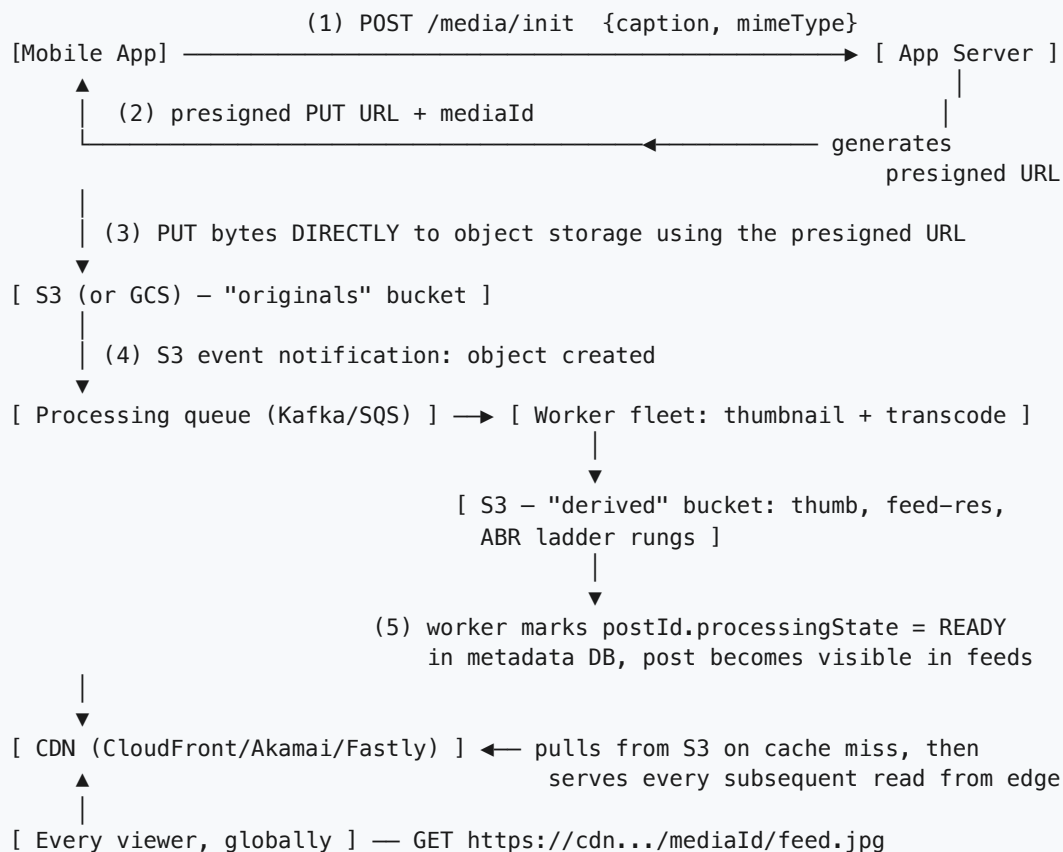
Why it dies, fast:

1. **Uploading through the app server** means every large file's bytes flow through your compute tier — a 50MB video ties up an app server thread/connection for the whole upload over a slow mobile network. At 1,160 uploads/sec, app servers become a bandwidth-bound proxy for no reason.
2. **BLOBs in Postgres** bloat the database, wreck buffer-cache hit rates for the *metadata* queries that actually need to be fast, make backups enormous, and don't get you a CDN — you'd still have to serve reads from your own servers.
3. **Synchronous thumbnailing** blocks the user's upload response on CPU-heavy image/video work — a multi-minute 4K video's transcode alone can take tens of seconds to minutes.

This naive design fails on **write path bandwidth**, **storage cost/design**, and **read-path latency/scale** simultaneously. Every fix below addresses one of those three.

4. The real V1 — direct-to-storage upload, async processing, CDN reads

Candidate: The fix is to take the app server **out of the byte-moving business entirely**, both for uploads and for reads.



Two decisions do all the work here: - **Presigned upload URLs**: the app server issues a short-lived, scoped credential (a presigned S3 PUT URL) and gets out of the way; bytes go client → S3 directly, over whatever network path is fastest, with S3 handling multi-part upload and resumability for large files. - **CDN in front of S3 for reads**: the CDN, not any of our servers, absorbs the 870k reads/sec, cached at edge locations near the viewer. Our origin (S3) only ever serves cache misses.

5. Scaling up — "now 100M DAU," then "now global," then multi-region

Interviewer: Good bones. Now scale it: what changes at 10x the users, and what changes when users are on every continent?

Read path (the 870k/s side) — this is mostly a CDN cache-hit-rate problem, not a compute one: - Popular posts (a celebrity's photo) get near-100% CDN hit rates — marginal cost per view is tiny. - The long tail (a photo three friends viewed) still benefits from **regional CDN edge PoPs** and **origin shielding** (one shield layer per region absorbs the "first viewer in this region" miss so origin sees one request, not thousands of simultaneous near-duplicate misses). - Metadata reads (captions, like counts, who-posted-what) are cached in Redis in front of the metadata database — small, hot, and separate entirely from the media bytes themselves.

Write path (the ~1,160 uploads/sec side) — mostly about the processing fleet, not storage: - S3 itself scales elastically — we don't "shard" object storage ourselves. - The **processing queue** and **worker fleet** are what need horizontal scale: partition the queue by `mediaId` so ordering per asset is preserved, and autoscale workers on queue depth. Video transcoding is the CPU-heavy tail — route video jobs to a GPU/CPU-optimized worker pool separate from the lightweight thumbnail pool so a backlog of 4K video jobs never starves simple photo thumbnailing.

Multi-region: - **Object storage:** each region writes to its nearest S3 region, then **cross-region replication** copies the original to at least one other region for durability and disaster recovery. The CDN pulls from whichever origin/replica is closest — origin choice is a CDN routing concern, not something the app needs to manage per-request. - **Metadata:** the post's row (owner, caption, storage keys, processing state) is written in the user's home region and replicated; feed/graph reads elsewhere may see it a few hundred ms later — acceptable, since even News Feed fan-out is already eventually consistent (see doc 04's fan-out discussion for the full argument). - **Uploads:** presigned URLs point at the region closest to the uploading client (via geo-routed API endpoint), so the slow, bandwidth-heavy leg — the client's own upload — never crosses an ocean if we can avoid it.

6. API + data model

```
POST /media/init          -> {mediaId, uploadUrl, uploadFields}    (get a presigned upload)
POST /media/{id}/complete -> {}                                  (client confirms upload done)
GET  /media/{id}          (metadata + CDN URLs)
POST /media/{id}/like | /media/{id}/comment
POST /stories             (like /media/init + ttl=24h)
GET  /feed?cursor=...      (delegates to feed service)
```

Media metadata — a row per asset, NOT a blob:

```
media_assets
  media_id      PK
  owner_id      (indexed)
  caption
  media_type     PHOTO | VIDEO
  storage_key    -- pointer into S3, e.g. "originals/{mediaId}/orig.mp4"
  derived_keys   -- {thumb, feed, abr_360p, abr_720p, abr_1080p}
  processing_state UPLOADING | PROCESSING | READY | FAILED
  created_at
  expires_at     -- null for posts; now()+24h for Stories

likes(media_id, user_id, created_at)      -- PK (media_id, user_id): dedup "like twice"
comments(comment_id, media_id, user_id, text, created_at)
story_views(story_id, viewer_id, viewed_at) -- PK (story_id, viewer_id): dedup views
```

Why a relational/document metadata store, not cramming media into it: the metadata rows are small (a few hundred bytes) and need **secondary access patterns** — "all media by this user," "is this post ready," foreign-key-ish joins to users — that a KV store handles awkwardly and that a relational or wide-column store (Postgres, sharded by `owner_id`, or DynamoDB/ Cassandra at extreme scale) handles cleanly. The **actual pixels/frames never enter this database** — they live only in S3, referenced by `storage_key`. Keeping bytes and metadata in separate systems, each doing what it's good at, is the single most important modeling decision in this whole design.

Likes/comments counters: don't `COUNT(*)` a growing table on every read. Keep a denormalized counter (`like_count` on the post row, or a Redis counter) updated via an async worker consuming a `likes` change stream — same "async off the critical path" pattern as the burger-giveaway grant log, just for a much lower-stakes number. A user's like click returns instantly against Redis; the durable row and the aggregate count catch up moments later.

7. Latency, consistency, async, caching

Latency — where the milliseconds go: - Upload: dominated by the client's own network (uploading 50MB on cellular), not our infra — which is exactly why direct-to-S3 (parallel multi-part chunks) beats proxying through one app-server TCP connection. - Read: dominated by **CDN cache hit vs. miss**. A hit is single-digit ms from a nearby edge; a miss pays a round trip to origin. Optimize for hit rate (long TTLs on immutable derived assets — a photo's `feed.jpg` never changes once generated, so cache it *forever* and mint a new key on any change) rather than trying to speed up misses.

Consistency — what's strong, what's eventual: - **Strong:** "did my upload succeed" (client gets an explicit ACK from S3's PUT + our `complete` call) and "is this my post" (owner-scoped writes). -

Eventual, and fine: like/comment/view counters, cross-region metadata replication, CDN cache freshness, feed fan-out (again, see doc 04). Nobody notices a like count that's 2 seconds stale; everybody notices a photo that fails to upload.

Async — off the critical path: - Thumbnail/transcode generation (the whole point of §9 below). - Counter aggregation, notification delivery ("X liked your photo"), search-index updates, Story-expiry cleanup (a TTL-driven job or S3 lifecycle rule that deletes expired Story originals/derivatives after 24h rather than a synchronous delete).

Caching — what and where: - **CDN** caches the actual media bytes (thumbnails, feed images, video segments) — by far the highest-value cache in the system given the 870k reads/sec. - **Redis** caches hot metadata (post details, like counts, "is this user's account private") in front of the metadata DB, invalidated on write (or short TTL, since a few seconds of staleness is acceptable here too). - **Client-side:** the app caches images it has already rendered so re-scrolling to a post doesn't even hit the CDN twice.

8. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Media bytes (photos/videos)	Object storage (S3)	Durable (11 9's), infinitely scalable, cheap per GB, integrates natively with a CDN. <i>Not a database BLOB column</i> — see Deep-dive A: it bloats the DB, kills cache locality for metadata, and gives you no CDN story.
Global delivery of media	CDN (CloudFront/Fastly) in front of S3	Caches at edge PoPs near viewers; absorbs ~870k reads/sec without touching origin on a hit. <i>Not "serve from app servers"</i> — app servers are compute, not a content-delivery network; they'd need to scale to viewer-count, not upload-count, and still be geographically far from most viewers.
Upload path	Presigned direct-to-S3 PUT (client → S3)	Removes the app server from the bandwidth path entirely; S3 handles multi-part/resumable upload natively. <i>Not proxying bytes through the app server</i> — ties up app-server capacity and bandwidth on every large upload for no benefit.
Async processing (thumbnail/transcode)	Queue (Kafka/SQS) + autoscaled worker fleet	Decouples slow, CPU-heavy work (video transcode can take minutes) from the upload response; workers scale independently and separately for video vs. photo jobs. <i>Not synchronous in-request processing</i> — the user would stare at a spinner, and one big transcode job would block the request thread.
Media metadata (captions, owners, storage keys, state)	Relational/wide-column store (Postgres sharded by owner, or Cassandra/DynamoDB at extreme scale)	Small structured rows, needs secondary access patterns (by user, by state); ACID for "did this post get created." <i>Not the same store as the bytes</i> — mixing them (BLOB columns) forces every metadata query to compete with giant binary I/O.
Like/comment/view counters	Redis counters, async-flushed to durable store	O(1) atomic increments absorb high write rates; a durable table underneath is the audit copy. <i>Not COUNT(*) on the comments/likes table per read</i> — that's an unbounded, ever-slower table scan.
Story expiry	TTL field + async sweeper / S3 lifecycle policy	Deletion happens off the critical path, in bulk, on a schedule. <i>Not per-request "check if expired and delete synchronously"</i> — wastes work on every read and races with concurrent readers.
Feed / fan-out	Delegated to the News Feed design (doc 04)	That problem (push vs. pull fan-out, celebrity fan-out) is orthogonal to media storage and

Concern	Choice	Why this, and why not the alternative
		already has a dedicated deep treatment — no need to re-derive it here.

The one-sentence justification you'd say out loud: *"Bytes live in S3 and are delivered through a CDN because that's a write-once-read-a-million-times problem that only object storage plus edge caching solves economically; metadata lives in a small, fast, ACID-ish store completely separate from the bytes; and everything CPU-heavy — transcoding, counting, expiry — happens asynchronously so the user's upload response is never gated on it."*

9. Failure modes & graceful degradation

Failure	What happens	Mitigation
Client upload interrupted mid-PUT (mobile network drop)	Partial object in S3	Multi-part upload lets the client resume from the last completed part, not restart from byte 0
Transcode worker crashes mid-job	Post stuck in PROCESSING	Queue redelivers after visibility timeout; job is idempotent (same output key), safe to redo
CDN edge PoP miss storm on a viral post	Many simultaneous misses hit origin at once	Origin shield / request coalescing collapses N simultaneous misses for the same key into 1 origin fetch
S3 region outage	Uploads/reads in that region degrade	Cross-region replication means CDN falls back to a replica region; uploads route to next-nearest healthy region
Metadata DB overloaded	Post creation slow/fails	Metadata write is small and fast normally; shed load by rejecting new uploads before rejecting reads (reads matter more, at 870k/s)
Counter service (Redis) blip	Like counts stale or briefly reset	Durable table is the source of truth; a reconciliation job resyncs Redis from Postgres/Cassandra periodically
Worker fleet backlog (viral moment + video-heavy uploads)	Thumbnails/transcodes delayed	See Deep-dive B — the post exists but isn't "ready" yet; feed logic must handle this gracefully, not crash

10. Follow-up curveballs

Interviewer: How do you avoid storing the same photo twice if two users upload the exact same viral meme image?

Candidate: Content-addressed dedup: hash the original bytes (SHA-256) before/after upload and check a `content_hash` -> `storage_key` index. If it already exists, point the new `media_assets` row at the

existing `storage_key` instead of writing new bytes — same trick as chunk-level dedup in a file-sync system, just applied at whole-object granularity here since photos aren't typically edited in place.

Interviewer: A user deletes a photo. What actually happens to the bytes?

Candidate: Soft-delete the metadata row immediately (it disappears from the API/feed right away — that's the user-visible contract). Actually deleting the S3 objects (original + derivatives) happens async, batched, after checking nothing else references that `storage_key` (relevant if we deduped above) — because a hard synchronous multi-object delete across regions on the request path adds latency for no user benefit.

Interviewer: How would you support "edit caption" without breaking cached CDN URLs?

Candidate: Caption lives in metadata (mutable, cheap), never in the image bytes or their URL. Since derived media URLs are **content/immutable-keyed** (a hash or version in the path), editing a caption never touches the CDN-cached bytes at all — only the metadata cache invalidates.

Interviewer: Stories need per-viewer "seen" tracking at scale — won't that table explode?

Candidate: `story_views(story_id, viewer_id)` only lives for ~24h by definition (TTL'd alongside the story itself), so it never grows unbounded the way permanent likes/comments do — size it for peak concurrent Stories × avg viewers, not for all-time volume, and let the same expiry job clean it up.

11. Deep-dive: "Why not store media as BLOBs in the database, or serve it from the app servers?"

Interviewer: Simplify this for me. Why not just keep the photo bytes in a `bytea` /BLOB column right next to the caption, and serve them straight from our own app servers? One less moving part.

Candidate: It's the natural first instinct — "it's all just data, why split it?" — but it breaks on nearly every axis that matters at this scale. Let me take the two ideas separately, since they fail for related but distinct reasons.

Why not BLOBs in the database?

Concern	BLOB column in Postgres/MySQL	Object storage (S3)
Cost per GB	Database storage (often backed by provisioned SSDs replicated for the whole DB) is far more expensive per GB than object storage tiers	S3 standard is a fraction of the cost, and cold/infrequent tiers (Glacier-class) cost even less
Buffer cache / working set	Every BLOB read/write competes for the same buffer cache and I/O bandwidth the <i>metadata</i> queries need	Metadata DB's working set stays small and hot; bytes never touch it

Concern	BLOB column in Postgres/MySQL	Object storage (S3)
	to stay fast — a burst of photo reads evicts the hot rows that power feed queries	
Backups	A full/incremental DB backup now moves petabytes of binary data alongside a few GB of actual structured data — backup/restore times explode	S3 has its own versioning/replication; DB backups stay fast because they're small
Horizontal scale	Sharding a relational DB to handle petabytes of blob growth is exactly the kind of operational pain relational stores are worst at	S3 scales storage capacity and throughput transparently, no sharding decisions for us
CDN integration	None — a CDN can't easily front "rows in a database"	Native — every major CDN treats S3 as a first-class origin
Replication for durability	Possible but couples media durability to database replication topology/cost	S3 offers 11 9's durability and cross-region replication as a built-in, decoupled feature

Why not serve reads straight from our own app servers?

Even if bytes lived in a dedicated blob store (not the DB) but we served them ourselves: - **We'd be building a worse CDN, badly, ourselves.** A CDN's entire job is edge caching close to viewers across the globe; replicating that with our own fleet means running servers in every region a CDN already has PoPs in — with none of the economies of scale. - **Read amplification would hit compute, not cache.** At 870k reads/sec, that's compute capacity (and its cost) scaling with *viewer* traffic instead of a CDN absorbing the vast majority as free cache hits. - **No natural edge caching = worse latency everywhere far from our servers.** A viewer in Mumbai fetching from a US-only app-server fleet pays a transoceanic round trip on every single image, every single time.

Rule of thumb: if a byte is written once and read many times by many different people in many different places, it belongs in **object storage behind a CDN**, full stop — never in a row-oriented database, and never served directly by stateful app compute. Databases are for structured, queryable, relatively small, frequently-mutated data; CDNs are for immutable-ish bytes read far more often than they're written, from everywhere.

12. Deep-dive: the upload-then-processing race — a photo appears before its thumbnail exists

Interviewer: Walk me through what happens end-to-end when someone uploads a *huge* video, in detail — and specifically, what happens if the feed tries to show that post before processing finishes?

Candidate: This is the sharpest edge case in the whole design, because it's not a rare failure — it's the **normal case** for any video and any large photo: processing always takes non-zero time, and the post's

existence (row created) and its readiness (derived assets exist) are two separate events with a gap between them.

The timeline

```
t=0ms      Client: POST /media/init {type: VIDEO, sizeHint: 80MB}
t=5ms      App server creates media_assets row: state=UPLOADING, returns presigned URL
t=5ms..40s Client uploads 80MB over cellular, multi-part, directly to S3
           (this alone can take 10s of seconds on a bad connection)
t=40s      Client: POST /media/{id}/complete
t=40s      App server: state -> PROCESSING; publishes {mediaId} to processing queue
t=40.1s    ▲ Post may ALREADY be eligible for fan-out into followers' feeds here,
           depending on how "complete" is wired to the feed pipeline –
           THIS is the race: state=PROCESSING, but no thumbnail/ABR ladder exists yet.
t=41s      A follower opens the app, feed service returns this post
           -> client tries to render media_assets.derived_keys.thumb
           -> KEY DOES NOT EXIST YET in the derived bucket
           -> broken image icon / 404 / infinite spinner
t=40s..75s Worker fleet: dequeues job, transcodes video into ABR ladder + thumbnail
           (35s of CPU-heavy work for an 80MB video is realistic)
t=75s      Worker writes derived assets to S3, sets state=READY
t=75s      NOW the post is actually safe to render
```

The gap between `t=40s` (post "exists," possibly already fanned out) and `t=75s` (post is actually renderable) is exactly where users see broken images — worse the bigger the video and the more backlogged the worker fleet is (e.g., right after a feature launch or viral moment when upload volume spikes and the same worker fleet that's already behind gets more jobs piled on).

Ranked fixes

1. **(Weak) Just show a spinner/placeholder client-side and poll.** The client polls `GET /media/{id}` until `state=READY`. Works, but polling at scale is wasteful, and a user scrolling past a "loading forever" post feels broken even if it eventually resolves.
2. **(Better) Don't fan the post out — or don't let the feed return it — until `state=READY`.** Gate visibility on processing completion: the post exists in the metadata store the moment it's uploaded (so the owner can see "processing..." on their own profile), but the **feed query filters out non-READY posts**, and the fan-out worker (from the News Feed design) only pushes to followers' feeds *after* the processing-complete event, not the upload-complete event. This closes the race at the source: nothing renders a post before it's renderable.
3. **(Best, layered on top of #2) Serve a "day-one" placeholder from data we already have.** S3 (or a lightweight in-request extraction) can produce a **tiny, cheap preview near-instantly** — e.g., a blurred low-res placeholder generated synchronously from the first uploaded bytes (a technique like BlurHash: encode a ~20-byte string representing a blurred version of the image, computed client-side or in a fast first-pass, stored directly on the `media_assets` row). The feed can render *that* immediately, then swap in the real thumbnail once `state=READY` pushes an update (via the same notification channel used for likes/comments). This gives users an immediate, non-broken visual instead of either a spinner or nothing.
4. **(Operational insurance) Prioritize the processing queue by asset size/type and add backpressure signaling.** Route small-photo thumbnail jobs (sub-second) through a separate, fast lane from multi-minute video transcodes, so a pile of videos never starves the (much more

common, much faster) photo case. If the queue depth crosses a threshold, surface that to the fan-out worker so it can widen the "don't fan out yet" window rather than racing ahead assuming processing is fast.

Recommendation: combine #2 and #3 — gate real feed visibility strictly on `state=READY` (so the race can never produce a broken/missing image), and layer a synchronous, cheap placeholder (BlurHash-style) on top so the *owner's own view* and any eager clients still get an immediate, polished visual rather than a blank tile while processing catches up. #4 is cheap insurance that shrinks how often the gap matters in the first place.

What to say out loud: *"The core race is that 'post exists' and 'post is renderable' are two different events separated by however long transcoding takes — which can be tens of seconds for video, and gets worse exactly when the worker fleet is most backlogged, i.e. during traffic spikes. The fix isn't to make processing instant; it's to never let anything downstream — feed fan-out, the API response — treat a post as visible until processing actually completes, and to fill that gap visually with a cheap synchronous placeholder like BlurHash instead of a broken image or a bare spinner."*

Summary — the mental model

Instagram's backend is fundamentally a **write-once, read-a-million-times, globally distributed bytes problem** wrapped around a comparatively small metadata layer. The winning moves: get media bytes out of the database and into **object storage behind a CDN** immediately — never proxy them through app servers, never store them as BLOBs; take uploads **directly client-to-S3** via presigned URLs so app servers never become a bandwidth bottleneck; do all CPU-heavy **thumbnailing and transcoding asynchronously** off a queue, and treat "processing complete" as a distinct, gated event so the feed never shows a post before its derived assets exist; and keep counters, captions, and other small mutable facts in a lightweight metadata store that never touches the bytes at all. The feed/fan-out mechanics on top of this are the News Feed problem (doc 04) — orthogonal, and already solved there.